

PostgreSQL Cheatsheet

Este documento recoge de manera resumida los comandos y conceptos más importantes de **PostgreSQL**.

Incluye ejemplos prácticos y comentarios explicativos.

1. Tipos de datos

```
-- Numéricos
SMALLINT, INTEGER, BIGINT, DECIMAL, NUMERIC, REAL,
DOUBLE PRECISION, SERIAL, BIGSERIAL, MONEY

-- Cadenas
CHAR(n), VARCHAR(n), TEXT

-- Fechas y horas
DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL

-- Booleanos
BOOLEAN -- TRUE / FALSE

-- UUID (requiere extensión pgcrypto)
UUID -- gen_random_uuid()

-- Binarios
BYTEA

-- JSON
JSON, JSONB

-- Especiales
ENUM, ARRAY, RANGE, TSVECTOR
```

2. CREATE (DDL) seguro

Uso de `IF NOT EXISTS` para evitar errores si el objeto ya existe.

Si no está soportado, se usa `OR REPLACE` o bloques `DO $$`.

```
CREATE TABLE IF NOT EXISTS usuarios (  
    id BIGSERIAL PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    email TEXT UNIQUE  
);  
  
CREATE INDEX IF NOT EXISTS idx_usuarios_email ON usuarios(email);  
  
CREATE SEQUENCE IF NOT EXISTS pedido_seq START 1;  
  
CREATE OR REPLACE VIEW clientes_activos AS  
    SELECT id, nombre FROM clientes WHERE activo;  
  
-- Crear base de datos solo si no existe  
DO $$  
BEGIN  
    IF NOT EXISTS (SELECT 1 FROM pg_database WHERE datname = 'mi_db') THEN  
        EXECUTE 'CREATE DATABASE mi_db';  
    END IF;  
END $$;
```

3. DML: INSERT, UPDATE, DELETE

```
-- Insertar registros
INSERT INTO usuarios (nombre,email)
VALUES ('Ana','ana@ejemplo.com');

-- Insertar desde SELECT
INSERT INTO clientes_activos (id,nombre)
SELECT id,nombre FROM usuarios WHERE activo;

-- Actualizar
UPDATE usuarios
SET activo = TRUE
WHERE id = 1;

-- Borrar registros
DELETE FROM usuarios WHERE activo = FALSE;

-- Vaciar tabla (más rápido que DELETE sin WHERE)
TRUNCATE TABLE usuarios;
```

4. SELECT

```
-- Selección básica
SELECT id, nombre
FROM usuarios
WHERE activo
ORDER BY nombre
LIMIT 10;

-- Agrupación
SELECT ciudad, COUNT(*)
FROM usuarios
GROUP BY ciudad
HAVING COUNT(*) > 5;

-- Subconsulta
SELECT *
FROM usuarios
WHERE id IN (
    SELECT usuario_id
    FROM pedidos
    WHERE total > 100
);

-- Condicional con CASE
SELECT nombre,
    CASE WHEN edad < 18 THEN 'Menor'
         WHEN edad <= 65 THEN 'Adulto'
         ELSE 'Mayor'
    END AS categoria
FROM usuarios;
```

5. JOINS

```
-- INNER JOIN: solo coincidencias
SELECT u.nombre, p.total
FROM usuarios u
INNER JOIN pedidos p
    ON u.id = p.usuario_id;

-- LEFT JOIN: todos los usuarios, aunque no tengan pedidos
SELECT u.nombre, p.total
FROM usuarios u
LEFT JOIN pedidos p
    ON u.id = p.usuario_id;

-- RIGHT JOIN / FULL OUTER JOIN
SELECT u.nombre, p.total
FROM usuarios u
RIGHT JOIN pedidos p ON u.id = p.usuario_id;

SELECT u.nombre, p.total
FROM usuarios u
FULL OUTER JOIN pedidos p ON u.id = p.usuario_id;

-- CROSS JOIN / USING
SELECT * FROM a CROSS JOIN b;
SELECT * FROM pedidos p
JOIN usuarios u USING (usuario_id);
```

6. Vistas y CTEs

```
-- Vista normal
CREATE OR REPLACE VIEW pedidos_detalle AS
    SELECT p.id, u.nombre, p.total
    FROM pedidos p
    JOIN usuarios u ON u.id = p.usuario_id;

-- Vista materializada (almacena resultados)
CREATE MATERIALIZED VIEW IF NOT EXISTS ventas_mes AS
    SELECT date_trunc('month', fecha) AS mes,
           SUM(total) AS total
    FROM ventas
    GROUP BY mes;

REFRESH MATERIALIZED VIEW ventas_mes;

-- CTE (vista temporal solo en la consulta)
WITH pedidos_mes AS (
    SELECT usuario_id, total
    FROM pedidos
    WHERE fecha >= date_trunc('month', now())
)
SELECT usuario_id, SUM(total) AS total_mes
FROM pedidos_mes
GROUP BY usuario_id;
```

7. Funciones incorporadas

```

-- Matemáticas
ABS(x), ROUND(x,n), POWER(x,y), RANDOM()

-- Fechas
NOW(), CURRENT_DATE, EXTRACT(YEAR FROM fecha), DATE_TRUNC('month', fecha)

-- Cadenas
LENGTH(), UPPER(), LOWER(), TRIM(), SUBSTRING(), CONCAT(), REPLACE()

-- JSONB
'{"a":1}':::jsonb->'a'
'{"a":1}':::jsonb->>'a'

-- Utilidades
COALESCE(), NULLIF(), GREATEST(), LEAST()

```

8. Funciones y Procedimientos (PL/pgSQL)

```

-- Función
CREATE OR REPLACE FUNCTION sumar(a INT, b INT)
RETURNS INT AS $$
BEGIN
    RETURN a+b;
END;
$$ LANGUAGE plpgsql;

-- Procedimiento
CREATE OR REPLACE PROCEDURE transferir(
    origen INT, destino INT, cantidad NUMERIC
) AS $$
BEGIN
    UPDATE cuentas SET saldo=saldo-cantidad WHERE id=origen;
    UPDATE cuentas SET saldo=saldo+cantidad WHERE id=destino;
END;
$$ LANGUAGE plpgsql;

CALL transferir(1,2,100);

```

9. Tablas en memoria

```
-- TEMP: solo durante la sesión
CREATE TEMP TABLE tmp_usuarios (
    id SERIAL,
    nombre TEXT
);
INSERT INTO tmp_usuarios (nombre)
VALUES ('Juan');

-- UNLOGGED: permanentes, sin WAL
CREATE UNLOGGED TABLE IF NOT EXISTS cache (
    clave TEXT PRIMARY KEY,
    valor JSONB,
    creado TIMESTAMPTZ DEFAULT now()
);
```

10. Transacciones

```
BEGIN;

    UPDATE cuentas
        SET saldo=saldo-100
        WHERE id=1;

    SAVEPOINT s1;

    UPDATE cuentas
        SET saldo=saldo+100
        WHERE id=2;

    -- Revertir parcialmente
    ROLLBACK TO SAVEPOINT s1;
COMMIT;

-- Cambiar nivel de aislamiento
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

11. Control de errores


```

-- Captura de excepción
DO $$
BEGIN
    INSERT INTO usuarios(id,nombre)
        VALUES (1,'Ana');
EXCEPTION WHEN unique_violation THEN
    RAISE NOTICE 'Usuario duplicado, ignorando';
END $$;

-- Ejemplo con GET DIAGNOSTICS
DO $$
DECLARE
    err_detail TEXT;
BEGIN
    PERFORM 1/0;
EXCEPTION WHEN division_by_zero THEN
    GET DIAGNOSTICS err_detail = PG_EXCEPTION_DETAIL;
    RAISE WARNING 'Error: %, detalle: %',
        SQLERRM, err_detail;
END $$;

```

Notas finales:

- Siempre que sea posible usa `IF NOT EXISTS` o `OR REPLACE` para evitar errores.
- Prefiere `JSONB` en lugar de `JSON` por eficiencia.
- Usa **transacciones** (`BEGIN/COMMIT/ROLLBACK`) en operaciones críticas.
- Los **CTEs** (`WITH`) son ideales para simplificar consultas complejas.